

Spring Data JPA — Quick Revision Notes

1. Persistence Layer after Hibernate & JPA

JPA/Hibernate remove manual SQL, mapping, and object creation work. But you still need a repository layer — services shouldn't touch EntityManager directly.

2. Repository Boilerplate Problem

Every entity (Student, Department, Course) needed its own repeated save/find/delete code. Only entity type + ID type actually change → so a generic `Repository<EntityType, IdType>` idea was born.

3. Spring Data vs Spring Data JPA

Spring Data = umbrella project for repository pattern across many DBs (JPA, MongoDB, Redis, etc). Spring Data JPA = the module specifically for JPA/Hibernate + relational DB.

4. JPA vs Spring Data JPA

JPA = spec (EntityManager, JPQL, annotations). Spring Data JPA = built on top of JPA, gives ready-made `JpaRepository` so you don't write implementation classes.

5. Repository Interface Hierarchy

- `Repository<T, ID>` → just a marker interface (no methods).
- `CrudRepository` → adds save/find/delete basics.
- `ListCrudRepository` → same but returns `List` instead of `Iterable`.
- `PagingAndSortingRepository` → adds sort/page methods (separate from CRUD since Spring Data 3).
- `JpaRepository` → combines everything (CRUD + paging + JPA-specific methods like flush).

6. Declaring a Repository Interface

Just extend `JpaRepository<Entity, IdType>` — no `@Repository` needed, no implementation class needed. Spring Boot auto-scans it.

7. Repository Proxy Internals

You can't `new` an interface. Spring creates a runtime **proxy** object that decides: is this a standard method (→ `SimpleJpaRepository`) or a custom/derived method (→ generate query)?

8. Important JpaRepository Methods

- `save()` → persist (new entity) or merge (existing/detached entity).
- `saveAll()` → loops `save()` internally, not guaranteed single batch insert.

- `findById()` → checks persistence context first, then DB.
- `findAll()` → loads everything — risky for big tables.
- `existsById()` / `count()` → lightweight existence/count queries.
- `delete()` / `deleteById()` / `deleteAll()` → remove entities.
- `flush()` → pushes pending changes to DB immediately (but doesn't commit transaction).
- `saveAndFlush()` → save + flush together, use only when you need immediate DB sync.

9. save() vs Dirty Checking

Inside a `@Transactional` method, if you just change a managed entity's field (no `save()` call), Hibernate auto-detects the change and runs UPDATE at commit — this is **dirty checking**. `save()` is mainly needed for new entities or merging detached ones.

10. Derived Query Methods

Spring Data reads method names like `findByEmail()` and auto-builds the query — no need to write JPQL. Format: `find/exists/count/delete + By + Condition` (e.g. `findByAgeGreaterThanAndActiveTrue`). Gets messy if the name becomes too long/complex — then use `@Query` instead.

11. Custom JPQL Queries (@Query)

Write your own JPQL using entity names (not table names) — e.g. `SELECT s FROM Student s WHERE s.email = :email`. Supports named (`:param`) and positional (`?1`) parameters. Joins use `s.department` (entity relationship), not `department_id`.

12. Native Queries

When you need real SQL (DB-specific functions/syntax), use `@Query(nativeQuery = true)` or `@NativeQuery`. Downside: tied to your specific database, less portable.

13. Sorting

DB doesn't guarantee row order without `ORDER BY`. Use `Sort.by("field").ascending()/descending()`. Can combine multiple fields. Never trust raw client input for sort field — validate against an allowlist first.

14. Pagination

Loading millions of rows at once = disaster (memory, network, performance). Pagination = fetch only a small "window" using `Pageable` / `PageRequest.of(pageNumber, pageSize, sort)`. Always add a unique tie-breaker (like `id`) in sort to keep page order consistent.

15. Page<T>

Contains both the data (`getContent()`) AND metadata — total elements, total pages, `isFirst/isLast/hasNext` etc. Needs an extra COUNT query to calculate totals.

16. Count Queries

Pagination internally runs 2 queries: one for actual page data (LIMIT/OFFSET) and one `COUNT(*)` for totals. For complex JPQL (joins/grouping), you can supply a custom `countQuery` in `@Query`.

17. Slice<T>

Like `Page` but lighter — no total count/pages, just knows if next page exists (`hasNext()`). No extra COUNT query needed. Best for "load more"/infinite scroll UIs; use `Page` when you need "Page 3 of 25" type info.

18. Complete Mental Model (Flow)

`Repository call` → `Proxy` → decides `inherited(SimpleJpaRepository)` or `derived/custom query` → `EntityManager` → `Hibernate generates SQL` → `DB executes` → `result mapped back to entity/DTO`.

Key Takeaways (Exam-Ready One-Liners)

1. JPA = spec, Hibernate = implementation, Spring Data JPA = repository abstraction on top of JPA.
2. `JpaRepository<Student, Long>` = entity type + ID type.
3. Spring generates a repository **proxy** at runtime — mostly handled by `SimpleJpaRepository`.
4. `save()` = persist (new) or merge (existing) depending on entity state.
5. Managed entities auto-update via **dirty checking**, even without `save()`.
6. Derived query methods are great until the method name gets too long — then use `@Query`.
7. JPQL = works on entity/field names; Native SQL = works on actual table/column names.
8. Always add explicit `Sort`/`ORDER BY` if order matters.
9. `Page` = data + totals (needs count query); `Slice` = data + hasNext only (lighter, no count query).
10. Spring Data JPA cuts boilerplate — but you still must understand transactions, entity states, fetching, and SQL performance yourself.